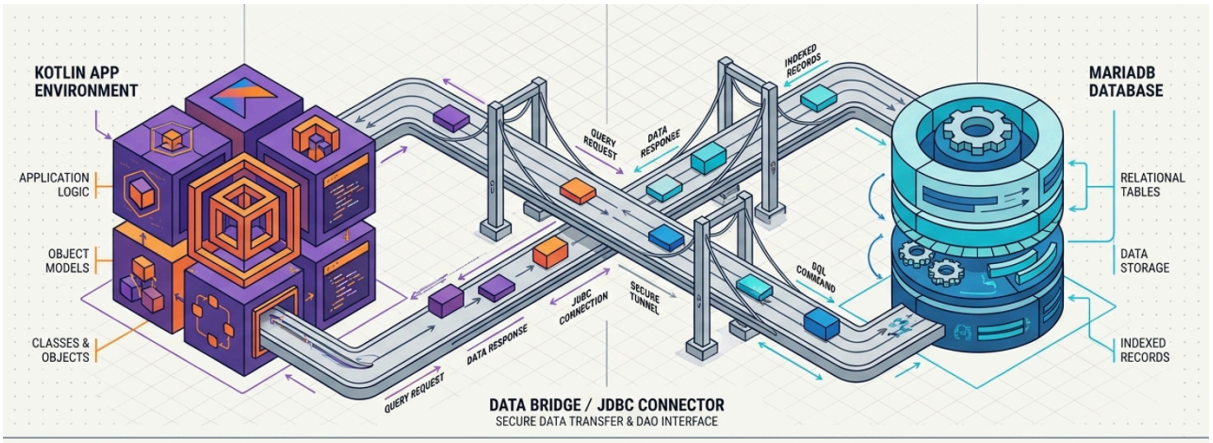
TP : JDBC et DAO



Construire le Pont de Données

Architecture, JDBC et Sécurité avec le Pattern DAO

Kotlin MariaDB JDBC Cybersécurité

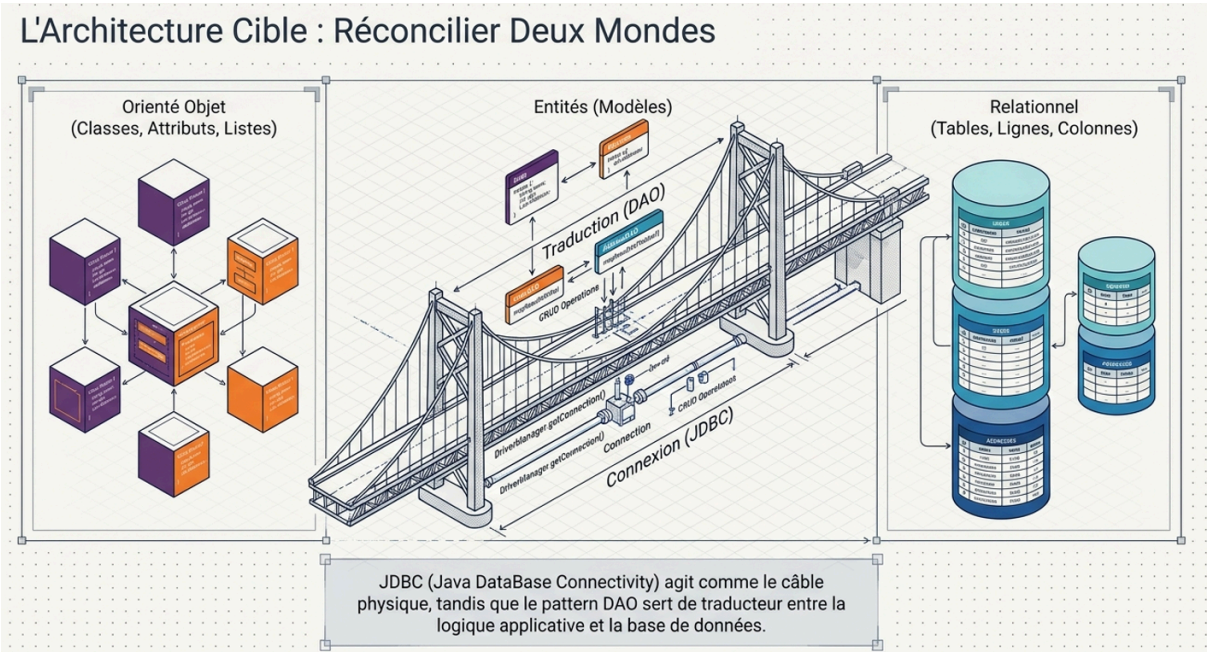
I. Introduction

JDBC (Java Database Connectivity) est une API qui permet à une application Java ou Kotlin de se connecter à un SGBD, d'y exécuter des requêtes SQL et d'en récupérer les résultats.

En PHP il existe une technologie similaire PDO.

Les objectifs principaux du TP sont :

- Se familiariser avec JDBC
- Comprendre le fonctionnement de requête préparé
- Apprendre à réaliser des opérations de types CRUD grâce au DAO
- Comprendre les risques liés aux injections SQL



II. Import de la BDD

Ouvrez un terminal, puis connectez-vous au SGBD.

Dans le terminal, tapez :

```
mariadb -u sio -p -h 172.17.70.X
```

ou

```
mysql -u sio -p -h 172.17.70.X
```

Remplacez X par votre numéro.

Le mot de passe est : **sio22**

Importation de la base de données

Téléchargez le fichier d'export SQL de la base de données : `bdd_jdbc.sql`.

Une fois connecté au SGBD, exécutez la commande suivante :

```
SOURCE /chemin/import.sql;
```

Remplacez `/chemin/` par le chemin absolu ou relatif du fichier `bdd_jdbc.sql`.

Vérification de la base de données

Tapez ensuite :

```
SHOW DATABASES;
```

Vérifiez que la base de données `bdd_jdbc` est présente.

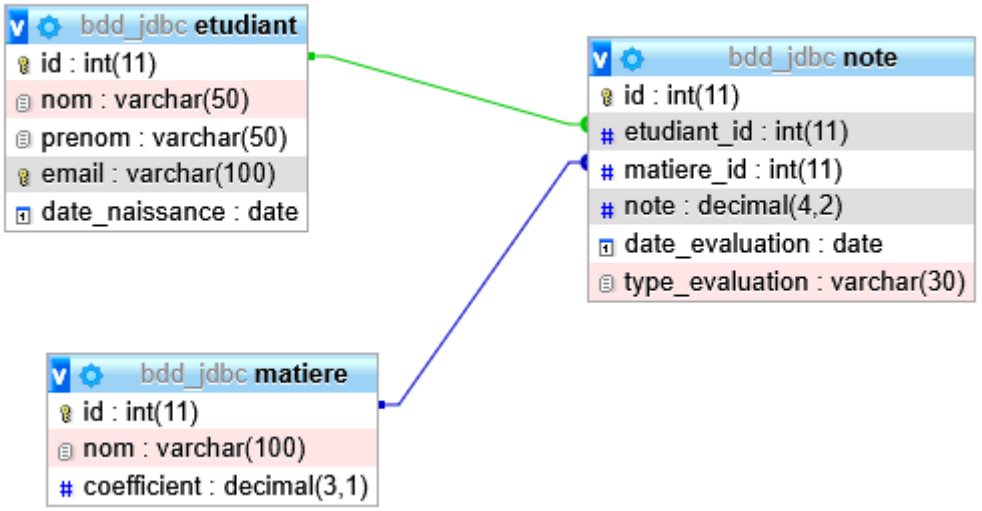
Sélectionnez ensuite la base de données avec :

```
USE bdd_jdbc;
```

Puis affichez les tables :

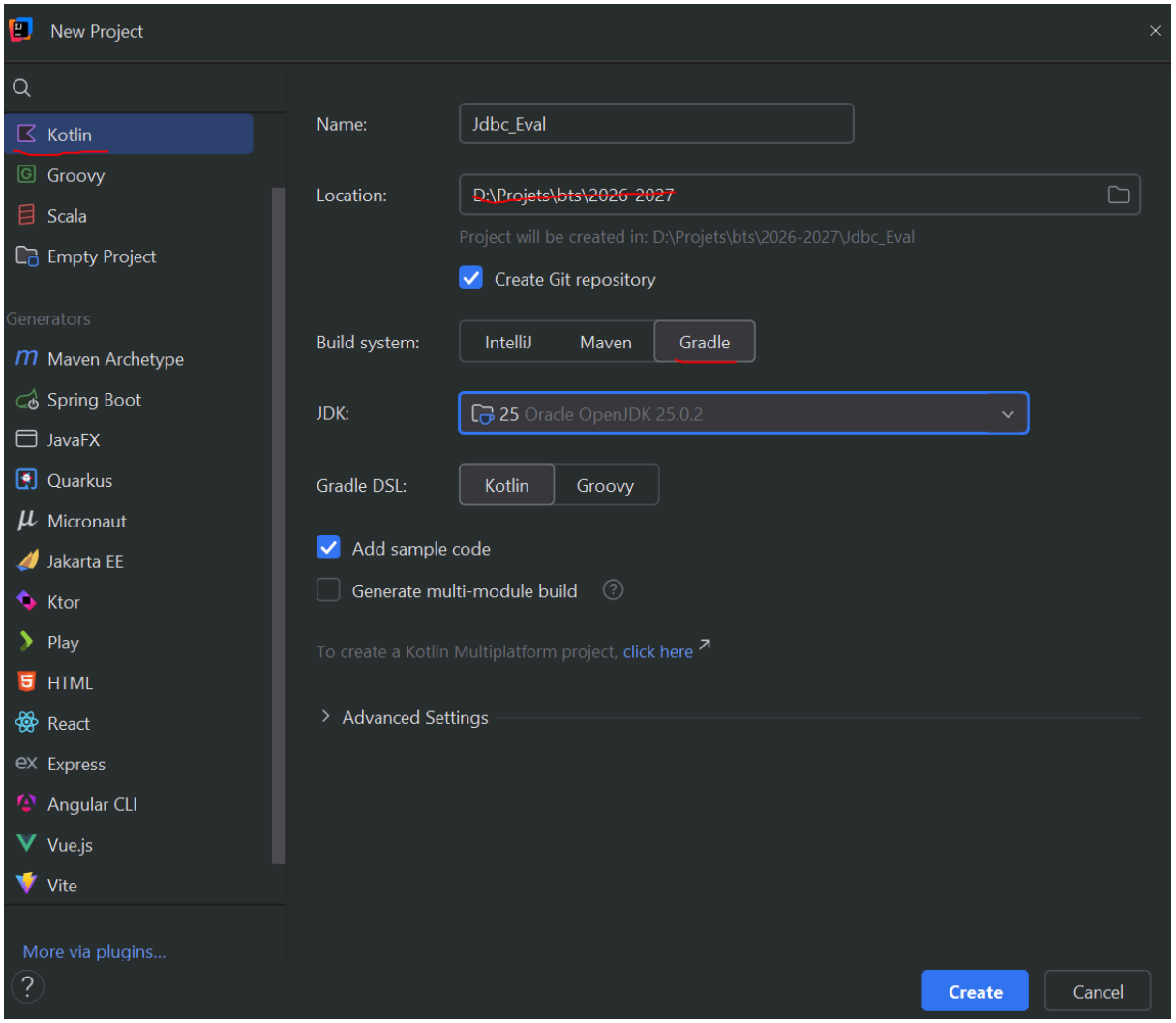
```
SHOW TABLES;
```

Vérifiez que les tables `etudiant`, `matiere` et `note` sont présentes.



III. Création du projet

Dans IntelliJ IDEA, créez un nouveau projet.



Vérifiez que le projet se lance correctement. Le premier lancement peut prendre un peu de temps.

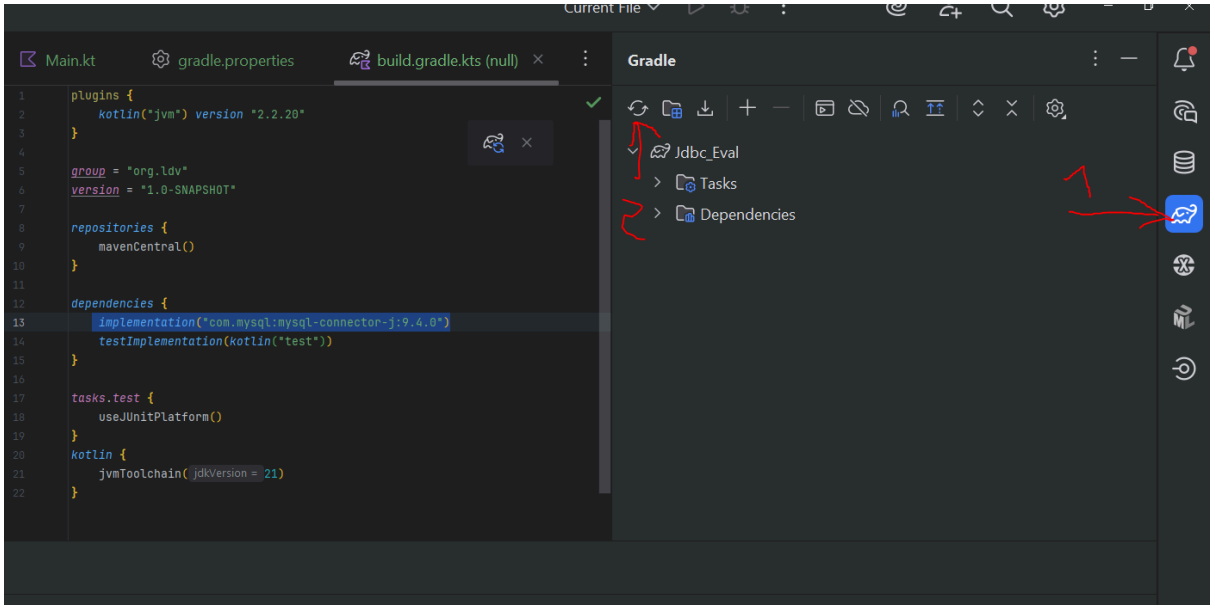
Ajouter une dépendance avec Gradle

Ajoutez maintenant une dépendance : un pilote JDBC permettant de communiquer avec le SGBD (MariaDB ou MySQL).

Ouvrez le fichier build.gradle.kts et dans dependencies ajouter la ligne suivante :

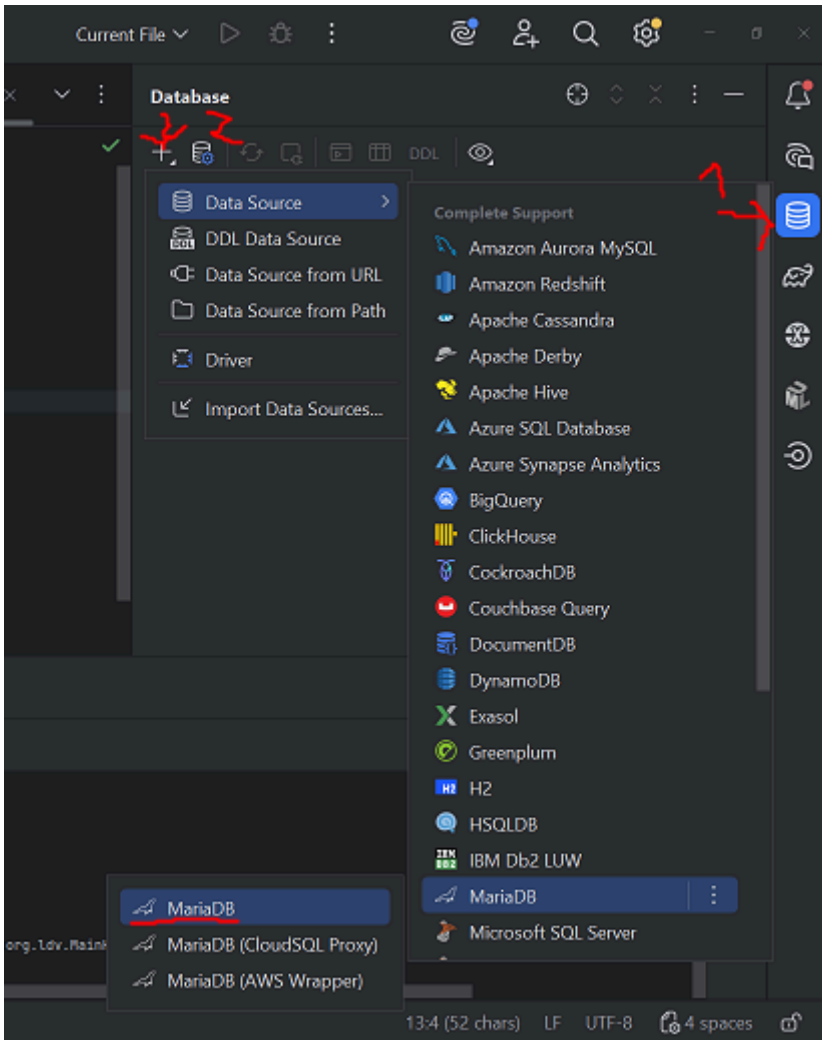
```
implementation("com.mysql:mysql-connector-j:9.4.0")
```

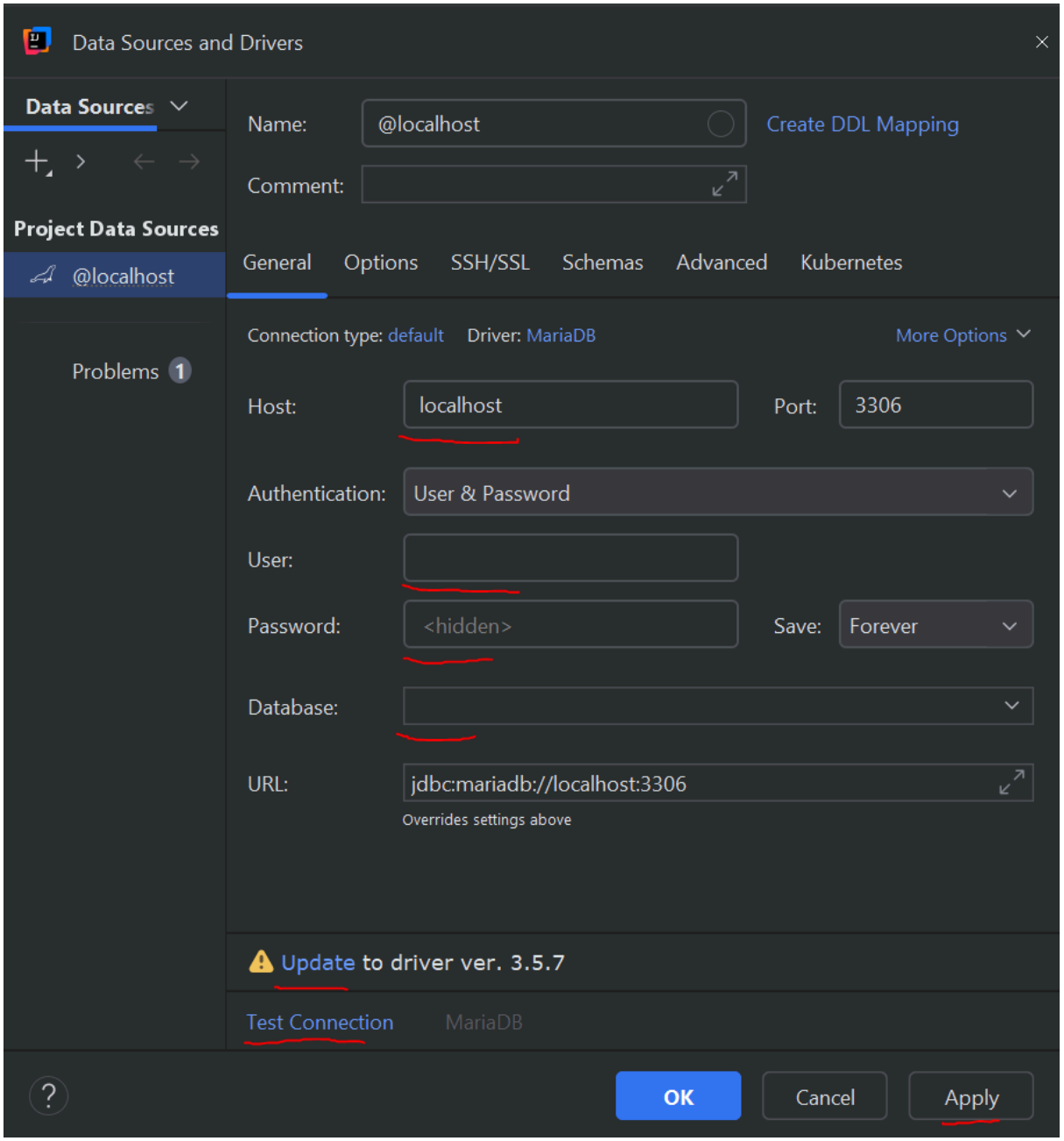
Demandez ensuite à Gradle, le gestionnaire de dépendances, de synchroniser le projet afin de télécharger cette bibliothèque.


Main.kt
gradle.properties
build.gradle.kts (null)
plugins {
kotlin("jvm") version "2.2.20"
}
group = "org.ldv"
version = "1.0-SNAPSHOT"
repositories {
mavenCentral()
}
dependencies {
implementation("com.mysql:mysql-connector-j:9.4.0")
testImplementation(kotlin("test"))
}
tasks.test {
useJUnitPlatform()
}
kotlin {
jvmToolchain(jdkVersion = 21)
}
Gradle
Jdbc_Eval
Tasks
Dependencies

Configurer une source de données (Data Source)

La fonctionnalité Data Source, disponible dans IntelliJ IDEA Ultimate, permet de visualiser et d'interroger la base de données, comme avec DBeaver ou phpMyAdmin.





Si l'option est proposée, mettez à jour le pilote.

Indiquez ensuite les informations :

Host : adresse IP du serveur de base de données

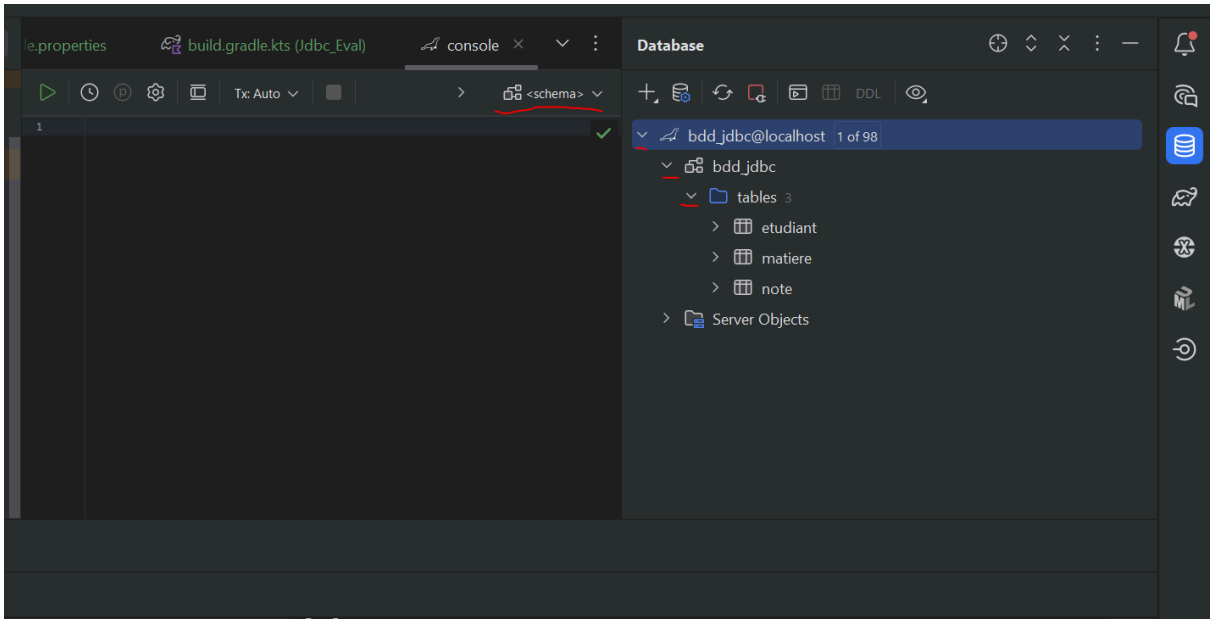
User : identifiant du SGBD (`sio`)

```
Password  = sio22
```

Database : `bdd_jdbc`

Testez la connexion. Si le test réussit, cliquez sur Apply.

Vous devriez obtenir ceci :



L'onglet de gauche est un endroit où vous pouvez taper des requêtes SQL si vous le souhaitez pour le configurer correctement je vous conseille de cliquer sur schéma et de choisir la bdd.

Publier le projet sur GitHub

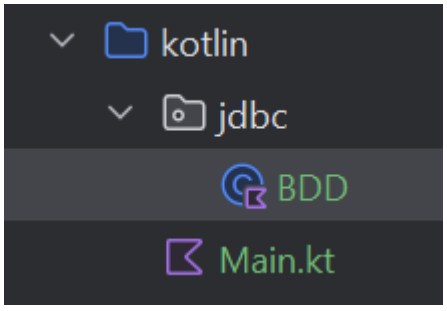
Créez un nouveau repository sur GitHub.

Publiez ensuite le projet dans ce dépôt.

IV. Utilisation de JDBC

Créer la classe `BDD`

Créez un package jdbc et dans le package ajoutez la classe BDD :



Dans la classe **laissez la première ligne : package …**

et remplacer le reste du code par le code suivant :

```
import java.sql.*



/**
* Cette classe gère la connexion à une base de données JDBC et l'exécution de
requêtes SQL préparées.
*
* @property url L'URL de la base de données.
```

```
* @property user Le nom d'utilisateur pour la connexion à la base de données.
* @property password Le mot de passe pour la connexion à la base de données.
* TODO : Remplacer localhost par l'adresse IP du serveur de BDD.
* TODO : le user et password par celui de la BDD.
*/
class BDD(

    var url: String = "jdbc:mysql://localhost:3306/bdd_jdbc",
    var user: String = "root",
    var password: String = "",
) {
    var connectionBDD: Connection? = null

    /**
     * Initialise une instance de la classe BDD et établit une connexion à la
base de données.
     */
    init {
        try {
            // Établir une connexion à la base de données lors de la création de
l'objet BDD
            this.connectionBDD = getConnection()
        } catch (erreur: SQLException) {
            // Gérer les exceptions liées à la connexion
            println("Erreur lors de la connexion à la base de données :
${erreur.message}")
        }
    }

    /**
     * Crée et retourne une connexion à la base de données.
     *
     * @return La connexion à la base de données.
     */
    fun getConnection(): java.sql.Connection? {
        //Chargement du driver
        Class.forName("com.mysql.cj.jdbc.Driver")
        // Créer et retourner une connexion à la base de données
        return DriverManager.getConnection(url, user, password)
    }

    /**
     * Exécute une requête SQL préparée et retourne le résultat sous forme de
ResultSet.
```

```
     *
     * @param preparedStatement La requête SQL préparée à exécuter.
     * @return Le résultat de la requête sous forme de ResultSet, ou null en cas
d'erreur.
     */
    fun executePreparedStatement(preparedStatement: PreparedStatement): ResultSet?
{
        try {
            // Exécuter la requête SQL préparée
            return preparedStatement.executeQuery()
        } catch (erreur: SQLException) {
            // Gérer les exceptions liées à l'exécution de la requête
            println("Une erreur est survenue lors de l'exécution de la requête :")
            println(preparedStatement.toString()) // Afficher la requête pour le
débogage
            println(erreur.message) // Afficher le message d'erreur
            return null // Renvoyer null en cas d'erreur
        }
    }

    /**
     * Finalise l'objet BDD en fermant la connexion à la base de données.
     */
    fun close() {
        // Fermer la connexion à la base de données lors de la finalisation de
l'objet BDD
        this.connectionBDD?.close()
    }
}
```

Traitez les éléments TODO indiqués dans les commentaires.

Lisez le code et les commentaires, puis essayez de comprendre le rôle de chaque partie.

Explications :

Cette classe permet de :

- Se connecter à la **BDD** grâce à la méthode **getConnection()**
- D'envoyer une requête SQL à la BDD (en utilisant la connexion) et retourner un résultat grâce à la méthode executePreparedStatement().
- De fermer la connexion quand on en a besoin pour économiser des ressources sinon la connexion peut rester ouverte côté **BDD**.

Dans le corps du constructeur on utilise tout de suite la méthode **getConnection()** et on stocke le résultat dans la propriété **connectionBDD**. La connexion peut échouer si les ***identifiants***, l'***IP*** ou le ***nom*** de la **BDD** sont incorrects et c'est pour cela que on fait un **try catch**.

Dans le try on écrit le code qui peut générer des erreurs et dans le catch on indique ce qui doit se passer s'il y une erreur (par exemple afficher un message), le catch a un paramètre qui représente l'erreur/exception ici il s'agit d'une erreur de type SQLException (il y a plein de sous type d'exception).

C'est la même logique quand on exécute la requête on le fait dans un ***try*** car la requête **SQL** peut générer une erreur au niveau du **SGBD**.

Pour tester si la configuration est correcte dans le fichier Main.kt

Remplacez la fonction main par ce code :

```
val db = BDD()

fun main() {
    db.getConnection()

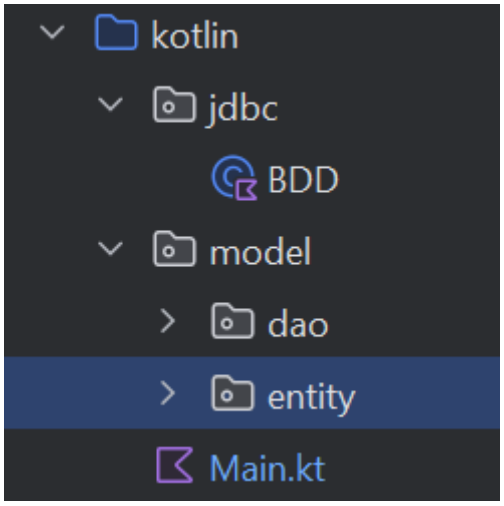
    db.close()
}
```

Puis exécutez le projet s'il n'y pas d'erreur alors votre configuration de la classe BDD est correcte. Sinon vous avez fait une erreur sur un des TODO.

V. Création des entités

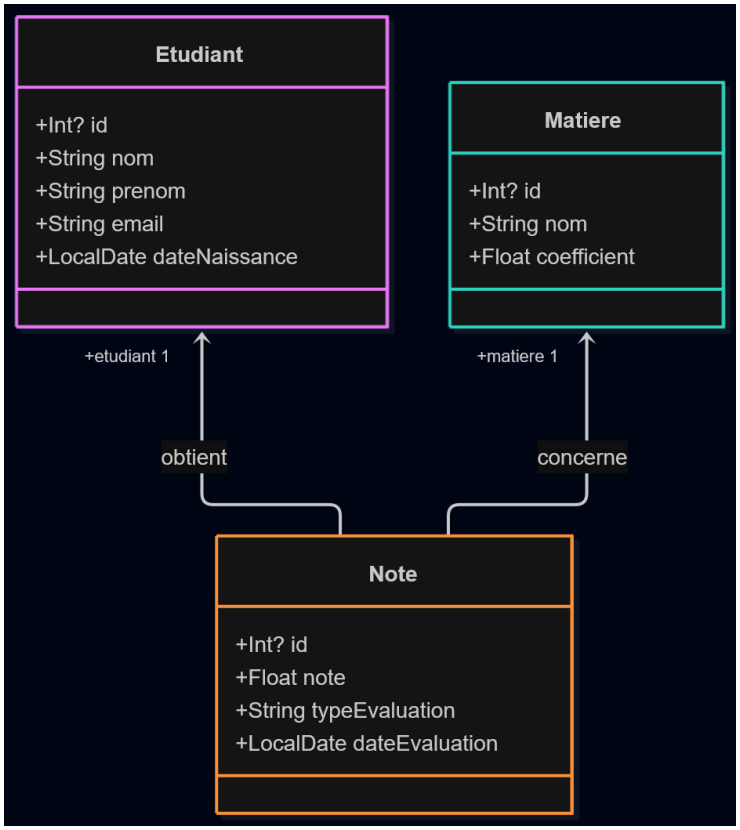
Nous allons maintenant créer le entités. Les entités (ou classes métiers) correspondent ici à des classes. Plus spécifiquement les classes qui correspondent à des tables de la BDD.

Créer les packages suivants :



A partir du diagramme de classe suivant créer les classes suivantes :

Note : On a choisi ici pour simplifier de réaliser des associations unidirectionnel afin d'éviter de rendre le code trop compliqué.



VI. DAO Etudiant

Définition : Un DAO ou Data Access Object est un type de composant (ici classe) qui communique avec la BDD en envoyant des requêtes SQL. Le résultat des requêtes est récupéré et traité/transformé par le DAO.

Chaque **DAO** correspond à une table et regroupe toutes les opérations **CRUD**.

Exemple : EtudiantDAO

Méthodes à implémenter :

- findAll() → récupère tous les étudiants de la table ;
- findById() → recherche un étudiant par son identifiant dans la table ;
- findByNom() → recherche un étudiant par son nom dans la table;
- save() → insère ou met à jour un étudiant dans la table;
- saveAll() → insère une liste d'étudiant dans la table;
- deleteById() → supprime un entraîneur de la table.

Notre objectif est maintenant de récupérer les étudiants de la base de données et de les transformer en objets de la classe `Etudiant`.

Pour cela, créez une classe `EtudiantDAO` dans le package `dao`.

```
class EtudiantDAO (val bdd: BDD = db) {

}
```

Nous allons tout de suite ajouter une méthode à la classe EtudiantDAO.

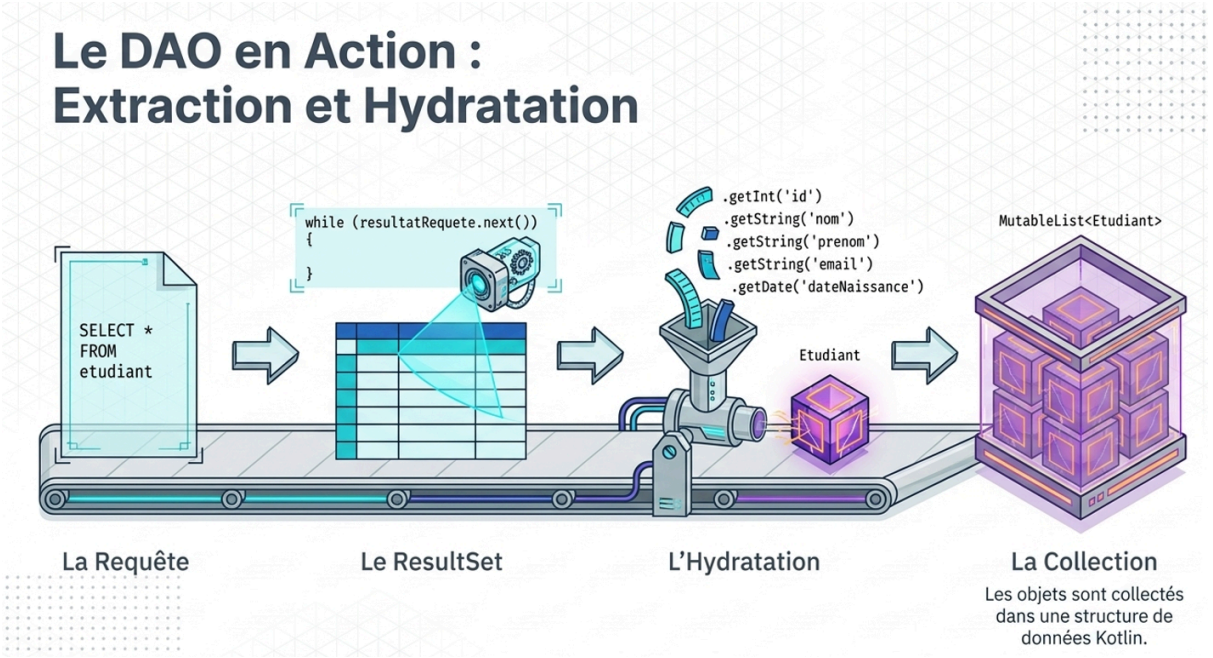
```
fun findAll(): MutableList<Etudiant> {
    val result = mutableListOf<Etudiant>()
    val sql = "SELECT * FROM etudiant"
    val requetePreparer = bdd.connectionBDD!!.prepareStatement(sql)
    val resultatRequete = bdd.executePreparedStatement(requetePreparer)

    if (resultatRequete != null) {
        while (resultatRequete.next()) {
            val id = resultatRequete.getInt("id")
            val nom = resultatRequete.getString("nom")
            val prenom = resultatRequete.getString("prenom")
            val email = resultatRequete.getString("email")
            val dateNaissance = resultatRequete.getDate("date_naissance")

result.add(Etudiant(id,nom,prenom,email,dateNaissance.toLocalDate()))
        }
    }

    requetePreparer.close()
    return result
}
```

Essayez de comprendre la méthode findAll() et ajouter des commentaires. Si besoin vous pouvez demander à vos professeurs ou à une IA



Modifiez `Main.kt`, puis lancez le projet :

```
val db = BDD()
val etudiantDAO = EtudiantDAO()
fun main() {
    db.getConnection()
    val etudiants = etudiantDAO.findAll()
    for (etudiant in etudiants) {
        println("${etudiant.id} => ${etudiant.nom}  ${etudiant.prenom}")
    }

    db.close()
}
```

Nous allons maintenant étudier un point important de cybersécurité : les risques liés à l'injection SQL. Prenons l'exemple d'une recherche d'étudiant par son nom.

La première version comporte volontairement une faille de sécurité.

Voici donc une première version de la méthode findByNom()

```
fun findByNom(nom: String): Etudiant? {
    var result: Etudiant? = null
    val sql = "SELECT * FROM etudiant WHERE nom = '$nom'"
    val requetePreparer = bdd.connectionBDD!!.prepareStatement(sql)
    val resultatRequete = bdd.executePreparedStatement(requetePreparer)


    if (resultatRequete != null && resultatRequete.next()) {
        val id = resultatRequete.getInt("id")
        val nom = resultatRequete.getString("nom")
```

```
        val prenom = resultatRequete.getString("prenom")
        val email = resultatRequete.getString("email")
        val dateNaissance = resultatRequete.getDate("date_naissance")
        result = Etudiant(id,nom,prenom,email,dateNaissance.toLocalDate())
    }


    requetePreparer.close()
    return result
}
```

J'ai simplement inséré dans requête SQL la variable nom.

Effectuez les trois tests suivants dans `main()` :

```
val test1 = etudiantDAO.findByNom("CARON-NAGEL")
println(" resultat = ${test1?.nom}    ${test1?.prenom}" )


val test2 = etudiantDAO.findByNom("TRUC")
println(" resultat = ${test2?.nom}    ${test2?.prenom}" )


val test3 = etudiantDAO.findByNom("' OR '1'='1")
println(" resultat = ${test3?.nom}    ${test3?.prenom}" )
```

A votre avis quelles sont les valeurs qui vont s'afficher dans la console pour les trois tests ?

Pourquoi est-ce que le test3 fonctionne ?

```
SELECT * FROM etudiant WHERE nom=2 OR 1=1
```

Quel mécanisme/faille de sécurité est exploité ici ?

Pour éviter ce problème nous allons modifier le code de la méthode findByNom() afin qu' il ne soit plus possible d'exploiter cette faille.

La faille est rendue possible à cause de la manière dont on injecte une variable dans la requête SQL.

```
fun findByNom(nom: String): Etudiant? {
    var result: Etudiant? = null
    val sql = "SELECT * FROM etudiant WHERE nom =?"
    val requetePreparer = bdd.connectionBDD!!.prepareStatement(sql)
    requetePreparer.setString(1, nom) // insere la valeur de l'id dans la
requete preparer
```

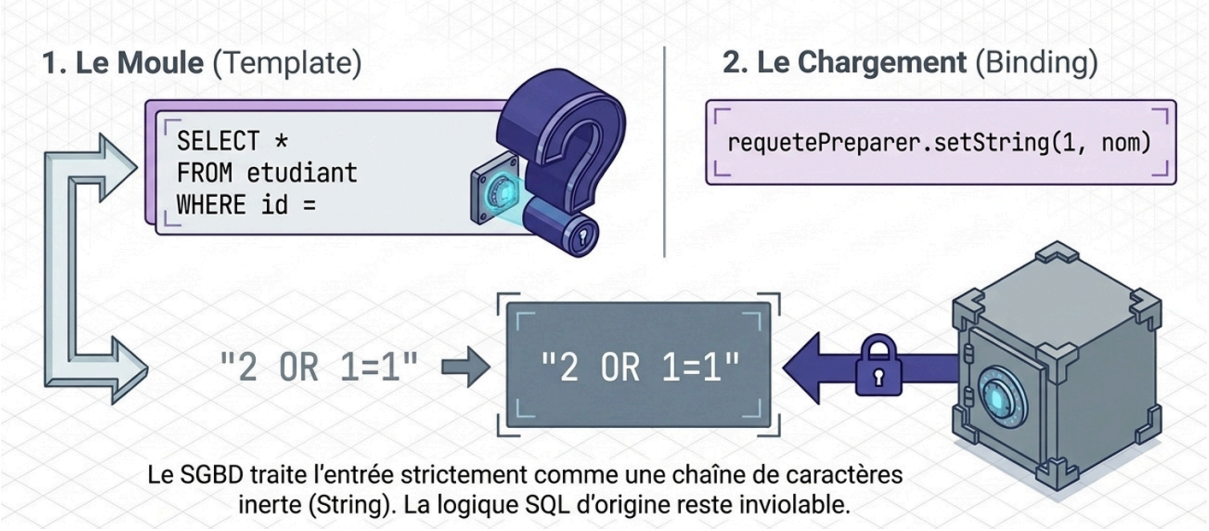
```
    val resultatRequete = bdd.executePreparedStatement(requetePreparer)

    if (resultatRequete != null && resultatRequete.next()) {
        val id = resultatRequete.getInt("id")
        val nom = resultatRequete.getString("nom")
        val prenom = resultatRequete.getString("prenom")
        val email = resultatRequete.getString("email")
        val dateNaissance = resultatRequete.getDate("date_naissance")
        result = Etudiant(id,nom,prenom,email,dateNaissance.toLocalDate())
    }

    requetePreparer.close()
    return result
}
```

Pour transmettre une valeur de manière sécurisée, placez un `?` à l'endroit correspondant dans la requête SQL préparée.

Pour indiquer la valeur que l'on souhaite injecter à la place ? On utilise ici la méthode setString().




```
requetePreparer.setString(1, nom) // associe la valeur de nom au paramètre
```

Le 1 correspond au 1er ? de la requête (s'il y en a plusieurs), nom est la valeur que l'on souhaite injecter à la place du ? en question.

Exécutez de nouveau les tests : le résultat du troisième test doit être différent.

	Concaténation Simple	PreparedStatement
Syntaxe d'insertion	`nom = $variable`	`nom = ?`
Sécurité (Injection)	**Vulnérable** (Code et données mélangés)	**Sécurisé** (Séparation stricte)
Typage des données	Manuel / Implicite	Sécurisé via `setInt()`, `setString()`, `setDate()`
Performance (Requêtes multiples)	**Faible** (Recompilé à chaque exécution)	**Élevée** (Plan d'exécution mis en cache par le SGBD)

CRUD

Voici les méthodes manquantes pour réaliser un CRUD Etudiant.

La méthode findById()

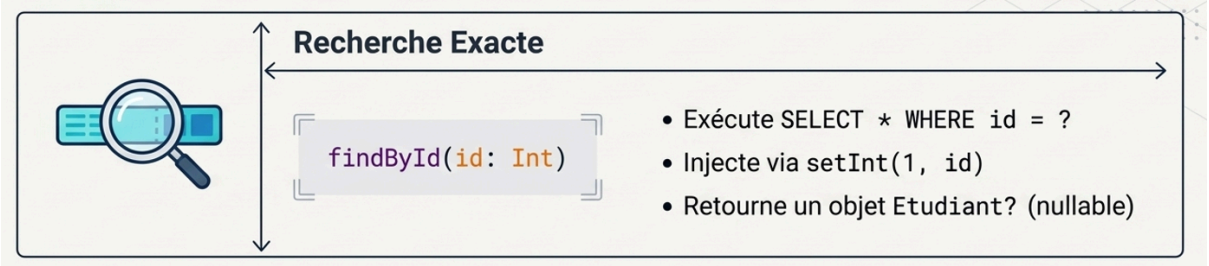
```
fun findById(id: Int): Etudiant? {
   var result: Etudiant? = null
   val sql = "SELECT * FROM etudiant WHERE id = ?"
   val requetePreparer = bdd.connectionBDD!!.prepareStatement(sql)
   requetePreparer.setInt(1, id) // insere la valeur de l'id dans la requete
preparer
   val resultatRequete = bdd.executePreparedStatement(requetePreparer)

   if (resultatRequete != null && resultatRequete.next()) {
       val nom = resultatRequete.getString("nom")
       val prenom = resultatRequete.getString("prenom")
       val email = resultatRequete.getString("email")
       val dateNaissance = resultatRequete.getDate("date_naissance")
       result = Etudiant(id,nom,prenom,email,dateNaissance.toLocalDate())
   }

   requetePreparer.close()
   return result
}
```

Cette méthode recherche un étudiant à partir de son identifiant. Elle retourne un objet `Etudiant`, ou `null` si aucun étudiant ne correspond à cet identifiant.

Comme l'identifiant est de type `Int`, utilisez `setInt()` plutôt que `setString()`.



La méthode save()

```
fun save(etudiant: Etudiant): Etudiant? {

    val requetePreparer: PreparedStatement

    if (etudiant.id == null) {
        // Insertion
        val sql = """
        INSERT INTO etudiant (nom, prenom, email, date_naissance)
        VALUES (?, ?, ?, ?)
    """.trimIndent()

        requetePreparer = bdd.connectionBDD!!.prepareStatement(
            sql,
            Statement.RETURN_GENERATED_KEYS
        )

        requetePreparer.setString(1, etudiant.nom)
        requetePreparer.setString(2, etudiant.prenom)
        requetePreparer.setString(3, etudiant.email)
        requetePreparer.setDate(
            4,
            java.sql.Date.valueOf(etudiant.dateNaissance)
        )

        val nbLigneMaj = requetePreparer.executeUpdate()

        if (nbLigneMaj > 0) {
            val generatedKeys = requetePreparer.generatedKeys

            if (generatedKeys.next()) {
                etudiant.id = generatedKeys.getInt(1)
            }

            generatedKeys.close()
            requetePreparer.close()
```

```
            return etudiant
        }

    } else {
        // Mise à jour
        val sql = """
        UPDATE etudiant
        SET nom = ?, prenom = ?, email = ?, date_naissance = ?
        WHERE id = ?
    """.trimIndent()

        requetePreparer = bdd.connectionBDD!!.prepareStatement(sql)

        requetePreparer.setString(1, etudiant.nom)
        requetePreparer.setString(2, etudiant.prenom)
        requetePreparer.setString(3, etudiant.email)
        requetePreparer.setDate(
            4,
            java.sql.Date.valueOf(etudiant.dateNaissance)
        )
        requetePreparer.setInt(5, etudiant.id!!)

        val nbLigneMaj = requetePreparer.executeUpdate()

        requetePreparer.close()

        if (nbLigneMaj > 0) {
            return etudiant
        }
    }

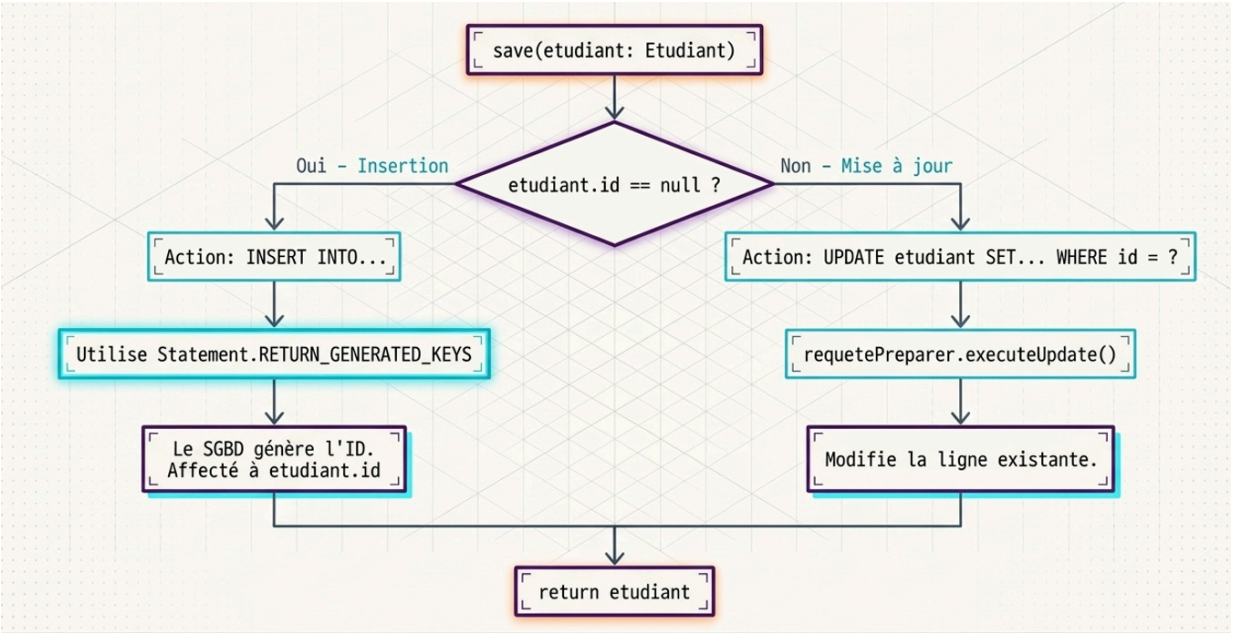
    requetePreparer.close()
    return null
}
```

De loin la méthode la plus complexe, elle reçoit en paramètre un objet de la classe Etudiant et on regarde si l'objet a un identifiant ou si celui-ci est null.

Si l'identifiant est nul, la méthode insère un nouvel étudiant avec une requête `INSERT INTO`.

Si l'objet possède déjà un identifiant, la méthode met à jour la ligne correspondante avec une requête `UPDATE`.

On retourne ensuite le nouvel objet étudiant (avec son id).

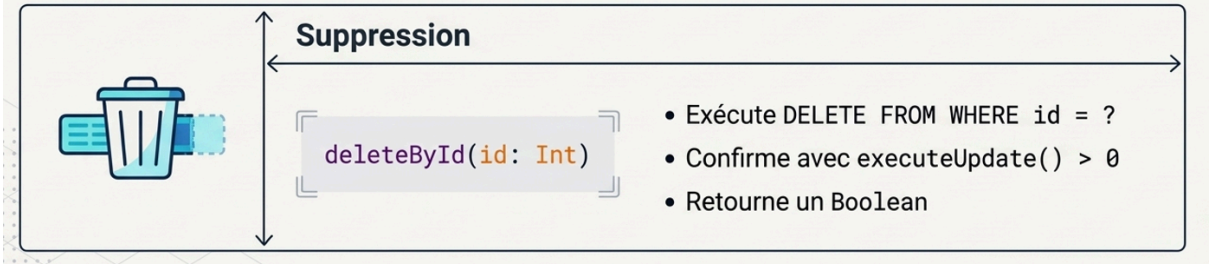

save(etudiant: Etudiant)
etudiant.id == null ?
Oui - Insertion
Non - Mise à jour
Action: INSERT INTO...
Action: UPDATE etudiant SET... WHERE id = ?
Utilise Statement.RETURN_GENERATED_KEYS
requetePreparer.executeUpdate()
Le SGBD génère l'ID.
Affecté à etudiant.id
Modifie la ligne existante.
return etudiant

La méthode deleteById()

```
fun deleteById(id: Int): Boolean {
    val sql = "DELETE FROM etudiant WHERE id = ?"
    val requetePreparer = bdd.connectionBDD!!.prepareStatement(sql)
    requetePreparer.setInt(1, id)

    return try {
        val nbLigneMaj = requetePreparer.executeUpdate()
        requetePreparer.close()
        nbLigneMaj > 0
    } catch (erreur: SQLException) {
        println("Erreur lors de la suppression de l'étudiant :
${erreur.message}")
        false
    }
}
```

Une méthode qui permet simplement de supprimer un utilisateur par son id.



La méthode saveAll()

```
fun saveAll(entraineurs: Collection<Etudiant>): MutableList<Etudiant> {
    val result = mutableListOf<Etudiant>()
    for (e in entraineurs) {
        val sauvegarde = save(e)
        if (sauvegarde != null) result.add(sauvegarde)
    }
    return result
}
```

Une méthode qui permet de sauvegarder une liste d'étudiants plutôt que de les sauvegarder un à un.

Tests fonctionnels

- Faites un test de findById() dans le main()
- Faites un test de save dans le cas d'un ajout dans le main()
- Testez `save()` pour modifier un étudiant dans `main()`.
- Testez `deleteById()` dans `main()`.

Puis mettre les tests en commentaires ou alors placer dans une nouvelle fonction test().

VI . Mise à jour du Main.kt

```
val db = BDD()
val etudiantDAO = EtudiantDAO()

fun main() {

    db.getConnection()


    afficherMenu()

    db.close()
}




fun afficherMenu() {

    var choix: String

    do {
        println()
        println("================================")
        println("     Application Evaluation")
        println("================================")
        println("s - Sélectionner un étudiant")
        println("a - Ajouter un étudiant")
        println("q - Quitter")
        println()

        choix = readln().lowercase()

        when (choix) {
            "s" -> selectionnerEtudiant()
            "a" -> ajouterEtudiant()
            "q" -> println("Au revoir !")
            else -> println("Choix invalide")
        }

    } while (choix != "q")
}


fun selectionnerEtudiant() {

    afficherEtudiants()

    println("Sélectionner un étudiant :")
    val id = readln().toInt()

    val etudiant = etudiantDAO.findById(id)

    if (etudiant == null) {
        println("Étudiant introuvable.")
        return
    }

    afficherEtudiant(etudiant)
```

```
    afficherMenuEtudiant(etudiant)
}


fun afficherEtudiants() {

    val etudiants = etudiantDAO.findAll()

    println()
    println("Liste des étudiants")
    println("--------------------")

    for (etudiant in etudiants) {
        println("${etudiant.id} => ${etudiant.nom} ${etudiant.prenom}")
    }
}


fun afficherEtudiant(etudiant: Etudiant) {

    println()
    println("Étudiant sélectionné")
    println("--------------------")

    println("Nom : ${etudiant.nom}")
    println("Prénom : ${etudiant.prenom}")
    println("Email : ${etudiant.email}")
    println("Date de naissance : ${etudiant.dateNaissance}")
}


fun afficherMenuEtudiant(etudiant: Etudiant) {

    println()
    println("----------------------------")
    println("1 - Supprimer")
    println("2 - Modifier")
    println("3 - Retour")

    val choix = readln()

    when (choix) {

        "1" -> supprimerEtudiant(etudiant)

        "2" -> modifierEtudiant(etudiant)

        "3" -> println("Retour au menu principal.")

        else -> println("Choix invalide.")
    }
}


fun supprimerEtudiant(etudiant: Etudiant) {

    val id = etudiant.id

    if (id == null) {
        println("Impossible de supprimer cet étudiant.")
        return
    }

    etudiantDAO.deleteById(id)
```

```
    println("Étudiant supprimé.")
}


fun modifierEtudiant(etudiant: Etudiant) {

    println("Nom :")
    etudiant.nom = readln()

    println("Prénom :")
    etudiant.prenom = readln()

    println("Email :")
    etudiant.email = readln()

    println("Date de naissance (AAAA-MM-JJ) :")
    etudiant.dateNaissance = LocalDate.parse(readln())

    etudiantDAO.save(etudiant)

    println("Étudiant modifié.")
}


fun ajouterEtudiant() {

    println()
    println("Ajout d'un étudiant")
    println("-------------------")

    println("Nom :")
    val nom = readln()

    println("Prénom :")
    val prenom = readln()

    println("Email :")
    val email = readln()

    println("Date de naissance (AAAA-MM-JJ) :")
    val dateNaissance = LocalDate.parse(readln())

    val etudiant = Etudiant(
        null,
        nom,
        prenom,
        email,
        dateNaissance
    )

    etudiantDAO.save(etudiant)

    println("Étudiant ajouté avec l'id ${etudiant.id}.")
}




fun test() {

//    val test1 = etudiantDAO.findByNom("CARON-NAGEL")
//    println(" resultat = ${test1?.nom}   ${test1?.prenom}" )
//
//    val test2 = etudiantDAO.findByNom("TRUC")
```

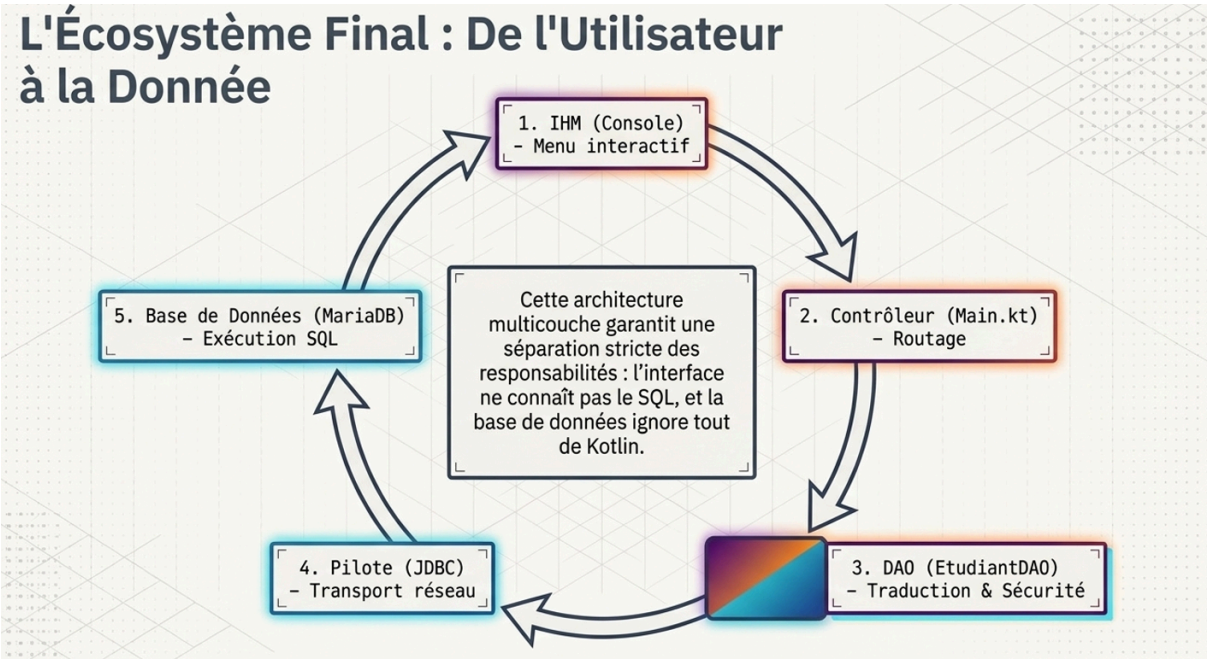
```
//    println(" resultat = ${test2?.nom}   ${test2?.prenom}" )
//
//    val test3 = etudiantDAO.findByNom("2 OR 1=1")
//    println(" resultat = ${test3?.nom}   ${test3?.prenom}" )

   //Test findById
   var copie = etudiantDAO.findById(1) !!
   println("copie = ${copie.nom}   ${copie.prenom}" )
   //Test save Insertion
   copie.id=null
   copie.prenom="kenny"
   copie.nom="MCCORMICK"
   copie.email = "test@email.com"

   copie = etudiantDAO.save(copie)!!
// Test save Update
   var kenny = etudiantDAO.findById(copie.id!!) !!
   println("kennyT0 = ${kenny.nom}   ${kenny.prenom} ${kenny.email}" )
   kenny.email="kenny.mccormick@southpark.com"
   kenny = etudiantDAO.save(kenny)!!
   println("kennyT1 = ${kenny.nom}   ${kenny.prenom} ${kenny.email}" )

   //test delete
   etudiantDAO.deleteById(copie.id!!)
   val kenny2 = etudiantDAO.findById(copie.id!!)
   println("kennyT3 = ${kenny2?.nom}   ${kenny2?.prenom} ${kenny2?.email}" )
}
```

Faites des tests en lançant l'application.



VII. Réalisation d’un CRUD Note

Créer des nouveaux DAO pour les notes et matières en vous inspirant de EtudiantDAO.

- Implémentez `findById()` et `findAll()` dans `MatiereDAO`.
- Implémentez toutes les opérations CRUD dans `NoteDAO`.

Dans la méthode afficheMenuEtudiant() ajouter les fonctionnalités suivantes :

- Afficher les notes de l’étudiant sélectionné
- Ajouter une note pour l’étudiant (il faut pouvoir consulter la liste des matières).
- Modifier une note de l’étudiant (il faut pouvoir consulter la liste des matières).
- Supprimer une note pour l’étudiant